

Blatt 10

Software Qualität

Test-Doubles: Stub, Mock und Spy

Luis Staudt

Überblick: Test-Doubles

Alle drei Aufgaben testen eine Klasse isoliert von ihrer Abhängigkeit. Dafür wird die echte Abhängigkeit durch ein *Test-Double* ersetzt, das über den Konstruktor injiziert wird (*Dependency Injection*). Die verwendeten Arten unterscheiden sich im Zweck:

- **Stub:** liefert fest verdrahtete Rückgabewerte. Es wird nur das Ergebnis (*state*) geprüft, nicht der Aufruf selbst.
- **Mock:** zeichnet die Interaktion auf. Der Test verifiziert, *ob, wie oft* und *mit welchen Argumenten* die Abhängigkeit aufgerufen wurde (*behavior*).
- **Spy:** umhüllt das echte Objekt, führt dessen Verhalten weiter aus und protokolliert zusätzlich die Aufrufe.

Als Mocking-Framework wird **Mockito** eingesetzt. Die Code-Abdeckung wurde mit der IntelliJ-Funktion *Run with Coverage* ermittelt.

Aufgabe 1: Verarbeitung von Bestellungen

Die Klasse `OrderService` bearbeitet eine Bestellung und delegiert die Zahlung an die Schnittstelle `IPaymentGateway`. Getestet werden beide Pfade der Methode `processOrder`: erfolgreiche Zahlung (Bestellung wird zurückgegeben) und fehlgeschlagene Zahlung (`null`).

Getestete Klassen

```
1 public interface IPaymentGateway {
2     boolean processPayment(double amount);
3 }
4
5 public class Order {
6     private double amount;
7     public Order(double amount) { this.amount = amount; }
8     public double getAmount() { return amount; }
9 }
10
11 public class OrderService {
12     private IPaymentGateway paymentGateway;
13
14     public OrderService(IPaymentGateway paymentGateway) {
15         this.paymentGateway = paymentGateway;
16     }
17
18     public Order processOrder(Order order) {
19         if (paymentGateway.processPayment(order.getAmount())) {
20             return order; // Bestellung erfolgreich
21         }
22         return null;      // Zahlung fehlgeschlagen
23     }
24 }
```

1.1 Stub

Der Stub liefert eine feste Antwort. Geprüft wird nur das Ergebnis von `processOrder`, der Aufruf selbst wird nicht verifiziert.

```
1 import org.junit.jupiter.api.Test;
2 import static org.junit.jupiter.api.Assertions.*;
3
4 public class OrderServiceStubTest {
5
```

```
6  static class PaymentGatewayStubErfolg implements IPaymentGateway {
7      public boolean processPayment(double amount) { return true; }
8  }
9
10 static class PaymentGatewayStubFehler implements IPaymentGateway {
11     public boolean processPayment(double amount) { return false; }
12 }
13
14 @Test
15 void erfolgreicheZahlungGibtBestellungZurueck() {
16     OrderService service = new OrderService(new PaymentGatewayStubErfolg());
17     Order order = new Order(100.0);
18     assertEquals(order, service.processOrder(order));
19 }
20
21 @Test
22 void fehlgeschlageneZahlungGibtNullZurueck() {
23     OrderService service = new OrderService(new PaymentGatewayStubFehler());
24     assertNull(service.processOrder(new Order(100.0)));
25 }
26 }
```

1.2 Mock ohne Framework

Der handgeschriebene Mock zählt die Aufrufe und merkt sich den übergebenen Betrag. So lässt sich die Interaktion verifizieren.

```
1 import org.junit.jupiter.api.Test;
2 import static org.junit.jupiter.api.Assertions.*;
3
4 public class OrderServiceManualMockTest {
5
6     static class PaymentGatewayMock implements IPaymentGateway {
7         private final boolean rueckgabe;
8         int aufrufZaehler = 0;
9         double letzterBetrag = Double.NaN;
10
11         PaymentGatewayMock(boolean rueckgabe) { this.rueckgabe = rueckgabe; }
12
13         public boolean processPayment(double amount) {
14             aufrufZaehler++;
15             letzterBetrag = amount;
16             return rueckgabe;
17         }
18     }
19
20     @Test
21     void processPaymentWirdMitRichtigemBetragAufgerufen() {
22         PaymentGatewayMock mock = new PaymentGatewayMock(true);
23         OrderService service = new OrderService(mock);
24
25         service.processOrder(new Order(100.0));
26
27         assertEquals(1, mock.aufrufZaehler);
28         assertEquals(100.0, mock.letzterBetrag, 0.0001);
29     }
30 }
```

```
29 }  
30 }
```

1.3 Mock mit Framework (Mockito)

Mit Mockito wird das Verhalten über `when().thenReturn()` festgelegt und die Interaktion über `verify()` geprüft.

```
1 import org.junit.jupiter.api.Test;  
2 import static org.junit.jupiter.api.Assertions.*;  
3 import static org.mockito.Mockito.*;  
4  
5 public class OrderServiceMockitoTest {  
6  
7     @Test  
8     void erfolgreicheZahlungGibtBestellungZurueck() {  
9         IPaymentGateway gateway = mock(IPaymentGateway.class);  
10        when(gateway.processPayment(100.0)).thenReturn(true);  
11        OrderService service = new OrderService(gateway);  
12        Order order = new Order(100.0);  
13  
14        assertEquals(order, service.processOrder(order));  
15        verify(gateway).processPayment(100.0);  
16    }  
17  
18    @Test  
19    void fehlgeschlageneZahlungGibtNullZurueck() {  
20        IPaymentGateway gateway = mock(IPaymentGateway.class);  
21        when(gateway.processPayment(anyDouble())).thenReturn(false);  
22        OrderService service = new OrderService(gateway);  
23  
24        assertNull(service.processOrder(new Order(100.0)));  
25        verify(gateway, times(1)).processPayment(100.0);  
26    }  
27 }
```

Aufgabe 2: Benutzerregistrierung

`UserService.registerUser` ruft `sendConfirmationEmail` der Schnittstelle `IService` auf. Da die Methode `void` zurückgibt, ist ein reiner Zustands-Test nicht möglich – es muss die Interaktion geprüft werden. Verifiziert wird, dass die Bestätigungs-E-Mail mit der richtigen Adresse versendet wird.

Getestete Klassen

```
1 public interface IService {
2     void sendConfirmationEmail(String email);
3 }
4
5 public class User {
6     private String name;
7     private String email;
8     public User(String name, String email) {
9         this.name = name; this.email = email;
10    }
11    public String getEmail() { return email; }
12 }
13
14 public class UserService {
15     private IService emailService;
16     public UserService(IService emailService) {
17         this.emailService = emailService;
18     }
19     public void registerUser(User user) {
20         emailService.sendConfirmationEmail(user.getEmail());
21     }
22 }
```

2.1 Mock ohne Framework

```
1 import org.junit.jupiter.api.Test;
2 import static org.junit.jupiter.api.Assertions.*;
3
4 public class UserServiceManualMockTest {
5
6     static class EmailServiceMock implements IService {
7         int aufrufZaehler = 0;
8         String letzteEmail = null;
9     }
```

```
10     public void sendConfirmationEmail(String email) {
11         aufrufZaehler++;
12         letzteEmail = email;
13     }
14 }
15
16 @Test
17 void registerUserSendetBestaetigungAnRichtigeAdresse() {
18     EmailServiceMock mock = new EmailServiceMock();
19     UserService service = new UserService(mock);
20
21     service.registerUser(new User("Anna", "anna@example.com"));
22
23     assertEquals(1, mock.aufrufZaehler);
24     assertEquals("anna@example.com", mock.letzteEmail);
25 }
26 }
```

2.2 Mock mit Framework (Mockito)

```
1 import org.junit.jupiter.api.Test;
2 import static org.mockito.Mockito.*;
3
4 public class UserServiceMockitoTest {
5
6     @Test
7     void registerUserSendetBestaetigungAnRichtigeAdresse() {
8         IEmailService emailService = mock(IEmailService.class);
9         UserService service = new UserService(emailService);
10
11         service.registerUser(new User("Anna", "anna@example.com"));
12
13         verify(emailService, times(1)).sendConfirmationEmail("anna@example.com");
14         verifyNoMoreInteractions(emailService);
15     }
16 }
```

Aufgabe 3: Applikation-Analytics

`AnalyticsService.trackEvent` ruft `logEvent` der Klasse `LoggingService` auf. Hier wird ein **Spy** verwendet: Im Gegensatz zum Mock führt der Spy das echte Verhalten weiter aus und protokolliert zusätzlich die Aufrufe. Gezeigt werden ein handgeschriebener Spy und ein Mockito-Spy.

Getestete Klassen

```
1 public interface ILoggingService {
2     void logEvent(String event);
3 }
4
5 public class LoggingService implements ILoggingService {
6     public void logEvent(String event) {
7         System.out.println("Ereignis protokolliert: " + event);
8     }
9 }
10
11 public class AnalyticsService {
12     private ILoggingService loggingService;
13     public AnalyticsService(ILoggingService loggingService) {
14         this.loggingService = loggingService;
15     }
16     public void trackEvent(String event) {
17         loggingService.logEvent(event);
18     }
19 }
```

Testklasse mit Spy

```
1 import org.junit.jupiter.api.Test;
2 import static org.junit.jupiter.api.Assertions.*;
3 import static org.mockito.Mockito.*;
4
5 public class AnalyticsServiceSpyTest {
6
7     // Handgeschriebener Spy: delegiert an das echte Objekt und zaehlt mit
8     static class LoggingServiceSpy implements ILoggingService {
9         private final LoggingService echt = new LoggingService();
10         int aufrufZaehler = 0;
11         String letztesEvent = null;
12     }
```

```
13     public void logEvent(String event) {
14         aufrufZaehler++;
15         letztesEvent = event;
16         echt.logEvent(event); // echtes Verhalten wird ausgefuehrt
17     }
18 }
19
20 @Test
21 void handgeschriebenerSpy() {
22     LoggingServiceSpy spy = new LoggingServiceSpy();
23     AnalyticsService service = new AnalyticsService(spy);
24
25     service.trackEvent("Login");
26
27     assertEquals(1, spy.aufrufZaehler);
28     assertEquals("Login", spy.letztesEvent);
29 }
30
31 @Test
32 void mockitoSpy() {
33     ILoggingService spy = spy(new LoggingService());
34     AnalyticsService service = new AnalyticsService(spy);
35
36     service.trackEvent("Login");
37
38     verify(spy, times(1)).logEvent("Login");
39 }
40 }
```

Code-Abdeckung

Alle 11 Testfälle laufen erfolgreich durch. Die getesteten Klassen werden vollständig abgedeckt; bei `OrderService` werden beide Zweige der `if`-Verzweigung (Zahlung erfolgreich / fehlgeschlagen) ausgeführt.

Klasse	Methoden	Zeilen	Zweige
OrderService	100 %	100 %	100 %
UserService	100 %	100 %	–
AnalyticsService	100 %	100 %	–